

 INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DE SÃO PAULO CAMPUS – São Paulo		
Disciplina: SPOWEB1		
Desenv. Web 1	Desafio 1: A Bilheteria (Controle de Lotação)	Data.: 09/05/2026
Nome do Aluno: Lucas Landes Xavier Martins		Prontuário: SP3193977
Professora: Claudete de Oliveira		
Link GitHub: https://github.com/Lucas-Landes-Xavier/Correcao-e-Adaptacao-Html-Css-JS-Lucas_Landes-DS213		

Atividade Prática: Auditoria e Integração de Sistemas

Contexto: O sistema de bilheteria do nosso "Festival de Inovação" foi desenvolvido às pressas e apresenta falhas críticas de lógica e sintaxe. Sua missão como programador sênior é auditar o código, corrigir os erros e criar uma interface funcional para o usuário.

Parte 1: O "Bug Hunt" (Caça aos Erros)

Enunciado: Abaixo está o código entregue pela equipe de desenvolvimento. Identifique e corrija os erros que impedem o funcionamento correto (dica: observe as chaves, o posicionamento dos comandos de interrupção e as mensagens exibidas).

// Código com Inconsistências – Bilheteria

```
let convidados = ["Rafaela", "Felipe", "SAIR", "Nicolas", "Amado", "Miguel"];
```

```
let entrada = [];
```

```
let i = 0;
```

```
while (entrada.length < 5) {
```

```
    console.log("Processando convidado: ", convidados[i]);
```

```
    if (convidados[i] == "SAIR") {
```

```
    }
```

```
    break;
```

```
        entrada.push(convidados[i]); i++;  
    }  
    console.log("Lotação esgotada!");  
    console.log(entrada);
```

Raciocínio Passo a Passo:

1. Foi criado um vetor contendo os nomes dos convidados.
2. Foi criado um vetor vazio chamado entrada, responsável por armazenar os convidados autorizados a entrar.
3. Foi criada a variável i, utilizada para percorrer as posições do vetor.
4. O laço de repetição executa enquanto o vetor entrada possuir menos de 5 pessoas.
5. Na primeira repetição, o programa processa o nome "Rafaela". Como o nome não é "SAIR", ele é adicionado ao vetor entrada.
6. Na segunda repetição, o programa processa o nome "Felipe". Como o nome também não é "SAIR", ele é adicionado ao vetor entrada.
7. Na terceira repetição, o programa processa o valor "SAIR". Nesse momento, a condição do if torna-se verdadeira e o comando break encerra imediatamente o laço de repetição.
8. Após o encerramento do laço, o programa exibe a mensagem "Lotação esgotada!".
9. Por fim, o vetor entrada é apresentado contendo apenas os nomes "Rafaela" e "Felipe".

Solução

```
C:\> Users > Lucalhas > Downloads > JS correcao.js > ...
1  let convidados = ["Rafaela", "Felipe", "SAIR", "Nicolas", "Amado", "Miguel"];
2  let entrada = [];
3  let i = 0;
4  while (entrada.length < 5) {
5      console.log("Processando convidado: ", convidados[i]);
6
7      // O erro no código original se tratava na posição do break, que estava fora do if, portanto,
8      //ele encerrava todo o laço de repetição, não permitindo a execução dos comandos que vinham após
9      //ele dentro do laço.
10     if (convidados[i] == "SAIR") {
11         break;
12     }
13
14     entrada.push(convidados[i]);
15     i++;
16 }
17 console.log("Lotação esgotada!");
18 console.log(entrada);
```

Print Screen

```
PS C:\Users\Lucalhas> node "c:\Users\Lucalhas\Downloads\correcao.js"
Processando convidado: Rafaela
Processando convidado: Felipe
Processando convidado: SAIR
Lotação esgotada!
[ 'Rafaela', 'Felipe' ]
PS C:\Users\Lucalhas>
```

Explicação Técnica

Foi criado um vetor chamado convidados, responsável por armazenar os nomes das pessoas que serão processadas pelo programa. Em seguida, foi criado o vetor entrada, inicialmente vazio, que receberá apenas os convidados autorizados a entrar. Também foi criada a variável i, utilizada como índice para percorrer as posições do vetor convidados.

O laço while executa enquanto a quantidade de elementos presentes em entrada for menor que 5. Durante cada repetição, o programa exibe no console o convidado que está sendo processado por meio da posição indicada pela variável i.

A estrutura condicional if verifica se o valor armazenado em convidados[i] é igual a "SAIR". Caso a condição seja verdadeira, o comando break é executado, interrompendo imediatamente o laço de repetição. Se a condição for falsa, o nome do convidado é

adicionado ao vetor entrada utilizando o método `push()`, e a variável `i` é incrementada para acessar a próxima posição do vetor.

O comando `break` não pode ficar fora da estrutura `if` porque ele encerra o laço independentemente de qualquer condição. Se ele fosse colocado logo após o bloco do `if`, como no exemplo apresentado, o `while` seria interrompido já na primeira repetição, mesmo que o convidado atual não fosse "SAIR". Dessa forma, o programa processaria apenas o primeiro elemento do vetor e encerraria o laço imediatamente, impedindo que os demais convidados fossem analisados. Portanto, o `break` deve permanecer dentro do `if`, garantindo que a interrupção do laço aconteça somente quando o valor "SAIR" for encontrado.

Parte 2: Desenvolvimento Front-End

Enunciado: Após corrigir a lógica no JavaScript, você deve:

1. HTML: Criar uma estrutura que contenha um título, um botão para "Iniciar Processamento" e uma lista (`<ul>` ou `<ol>`) para exibir os nomes validados.
2. CSS: Estilizar a página para que tenha uma aparência profissional (ex: botões destacados e lista organizada).
3. JavaScript (Integração): Transformar o código corrigido em uma função. Esta função deve ser disparada pelo clique no botão e, em vez de apenas mostrar no `console.log`, deve inserir os nomes dos convidados diretamente na lista da sua página HTML.

1 Código HTML

Raciocínio Passo a Passo:

1. O navegador começa lendo o documento HTML.
2. A tag `<!DOCTYPE html>` informa que o arquivo utiliza HTML5.
3. A tag `<html>` marca o início da estrutura da página.
4. Dentro do `<head>`, são carregadas informações importantes, como a codificação UTF-8, a configuração de responsividade para dispositivos móveis, o título da página e o arquivo CSS responsável pela estilização.
5. Quando o navegador entra no `<body>`, ele começa a exibir os elementos visuais da página.
6. A `<div class="container">` funciona como uma área principal que organiza todo o conteúdo do sistema.

7. O `<h1>` exibe o título “Lista da bilheteria”.
8. O `<input>` cria um campo onde o usuário pode digitar um nome.
9. O atributo `id="nome"` permite que o JavaScript localize esse campo posteriormente.
10. O atributo `placeholder` exibe um texto de ajuda enquanto o usuário ainda não digitou nada.
11. O `<button>` cria o botão “Processar”.
12. Quando o botão é clicado, o atributo `onclick` executa a função `validarPessoas()`.
13. O `<p id="mensagem">` é utilizado para mostrar mensagens ao usuário, como avisos, erros ou confirmações.
14. A `<div class="lista_container">` organiza visualmente a área destinada à lista de nomes validados.
15. O `<ul id="lista">` cria uma lista inicialmente vazia, que será preenchida dinamicamente pelo JavaScript.
16. No final do documento, a tag `<script src="script.js">` conecta o arquivo JavaScript responsável pela lógica do sistema.

Solução

```
C: > Users > Lucalhas > OneDrive > Área de Trabalho > IFSP > Técnico em Desenvolvimento de Sistemas > WEB > Bilheteria > idex.html >
1  <!DOCTYPE html>
2  <!-- Define que o documento utiliza HTML5 -->
3  <html lang="en">
4  <!-- Início do documento HTML e definição do idioma -->
5
6  <head>
7      <!-- Área de configurações da página -->
8      <meta charset="UTF-8">
9      <!-- Define o padrão de caracteres como UTF-8 -->
10     <meta name="viewport" content="width=device-width, initial-scale=1.0">
11     <!-- Configura a responsividade da página em diferentes dispositivos -->
12     <title> Auditoria e Integração de Sistemas</title>
13     <!-- Define o título exibido na aba do navegador -->
14     <link rel="stylesheet" href="styles.css">
15     <!-- Conecta o arquivo CSS responsável pela estilização -->
16 </head>
17
18 <body>
19     <!-- Corpo da página -->
20     <div class="container">
21         <!-- Div principal que agrupa os elementos da interface -->
22         <h1>Lista da bilheteria</h1>
23         <!-- Título principal da página -->
24
25         <input type="text" id="nome" placeholder="Digite um nome">
26         <!-- Campo de texto utilizado para digitar nomes -->
27
28         <button onclick="validarPessoas()">Processar</button>
29         <!-- Botão que executa a função validarPessoas() -->
30
31         <p id="mensagem"></p>
32         <!-- Área utilizada para exibir mensagens ao usuário -->
33
34         <div class="lista_container">
35             <!-- Div responsável pela área da lista -->
36             <h2>Nomes Validados</h2>
37             <!-- Subtítulo da lista -->
38             <hr>
39             <!-- Linha horizontal para separação visual -->
40             <ul id="lista">
41                 <!-- Lista não ordenada onde os nomes serão adicionados -->
42             </ul>
43         </div>
44     </div>
45
46     <script src="script.js"></script>
47     <!-- Conecta o arquivo JavaScript responsável pela lógica da página -->
48 </body>
49
50 </html>
```

Explicação Técnica

O código apresentado representa a estrutura de uma aplicação web simples desenvolvida em HTML, integrada com CSS e JavaScript. O HTML é responsável por definir a estrutura da página, o CSS controla a aparência visual e o JavaScript executa a lógica e a interatividade do sistema.

A estrutura do documento inicia com a declaração `<!DOCTYPE html>`, que informa ao navegador que a página utiliza a versão HTML5. Em seguida, a tag `<html>` delimita todo o conteúdo da página. O atributo `lang="en"` define o idioma principal do documento como inglês, auxiliando navegadores e mecanismos de busca.

Dentro da seção `<head>`, encontram-se os metadados da página. A tag `<meta charset="UTF-8">` define a codificação de caracteres, permitindo o uso correto de acentos e símbolos especiais. Já a tag `<meta name="viewport">` configura a responsividade da página, garantindo melhor adaptação em dispositivos móveis.

A tag `<title>` define o título exibido na aba do navegador. O elemento `<link rel="stylesheet" href="styles.css">` realiza a importação de um arquivo CSS externo, separando a estilização da estrutura HTML e facilitando a organização do projeto.

No corpo da página, a `<div class="container">` atua como um contêiner principal para organizar todos os elementos da interface. O título principal é exibido pela tag `<h1>`.

O elemento `<input type="text">` cria um campo de entrada de texto para que o usuário digite nomes. O atributo `id="nome"` serve como identificador único, permitindo que o JavaScript acesse esse elemento utilizando métodos como `document.getElementById()`.

O botão `<button>` possui o evento `onclick`, responsável por executar a função `validarPessoas()` quando o usuário clicar nele. Esse mecanismo conecta a interface HTML à lógica implementada no arquivo JavaScript.

O elemento `<p id="mensagem">` é utilizado para exibir mensagens dinâmicas ao usuário, como confirmações ou alertas. Seu conteúdo pode ser alterado via JavaScript utilizando propriedades como `innerHTML` ou `textContent`.

A estrutura `<ul id="lista">` representa uma lista não ordenada. Ela é utilizada para armazenar dinamicamente os nomes processados pelo sistema, permitindo que novos elementos `<li>` sejam adicionados conforme as validações realizadas.

Por fim, a tag `<script src="script.js"></script>` importa o arquivo JavaScript externo. Esse arquivo contém toda a lógica funcional do sistema, como validação de dados, manipulação de elementos HTML e atualização dinâmica da interface.

2 Código CSS

Raciocínio Passo a Passo:

1. O navegador carrega o arquivo CSS ligado ao HTML.
2. A regra `body` define a fonte, a cor de fundo e centraliza os elementos usando Flexbox.
3. Os seletores `h1` e `h2` alteram a cor dos títulos para branco.
4. A classe `.container` estiliza a caixa principal da aplicação, adicionando largura, cor de fundo, espaçamento interno, cantos arredondados e sombra.
5. A classe `.lista_container` estiliza a área onde os nomes validados serão exibidos.
6. O seletor `#titulo` altera a cor de um elemento específico com esse id.
7. A regra `input` personaliza o campo de texto, definindo borda, espaçamento e cantos arredondados.
8. A regra `button` estiliza o botão, alterando tamanho, cor, fonte e cursor.
9. A pseudo-classe `button:hover` modifica a cor do botão quando o mouse passa sobre ele.
10. A regra `p` estiliza os parágrafos usados para mensagens ao usuário.
11. A regra `ul` remove os marcadores da lista e centraliza os itens.

Solução

```
C: > Users > Lucalhas > Downloads > # styles.css > ul
1  body {
2      /* Define a fonte principal da página */
3      font-family: Arial, sans-serif;
4      /* Utiliza Flexbox para organizar os elementos */
5      display: flex;
6      /* Centraliza os elementos horizontalmente */
7      justify-content: center;
8      /* Centraliza os elementos verticalmente */
9      align-items: center;
10     /* Define a altura do body ocupando toda a tela */
11     height: 100vh;
12     /* Define a cor de fundo da página */
13     background-color: #1a192d;
14 }
15
16 h1, h2{
17     /* Define a cor dos títulos h1 e h2 */
18     color: white
19 }
20
21 .container {
22     /* Centraliza o texto dentro da div */
23     text-align: center;
24     /* Espaçamento interno da div */
25     padding: 20px;
26     /* Cor de fundo da caixa principal */
27     background-color: #4e6695;
28     /* Define a largura da caixa */
29     width: 400px;
30     /* Arredonda os cantos da div */
31     border-radius: 10px;
32     /* Adiciona sombra ao redor da caixa */
```

```

C: > Users > Lucalhas > Downloads > # styles.css > ul
21  .container {
33      box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
34  }
35
36  .lista_container{
37      /* Centraliza o texto dentro da div */
38      text-align: center;
39      /* Espaçamento interno */
40      padding: 20px;
41      /* Cor de fundo da área da lista */
42      background-color: #13042e;
43      /* Arredonda os cantos da caixa */
44      border-radius: 10px;
45      /* Adiciona sombra ao redor da caixa */
46      box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
47  }
48
49  #titulo{
50      /* Define a cor do título com id "titulo" */
51      color: #aliceblue;
52  }
53
54  input {
55      /* Espaçamento interno do campo */
56      padding: 10px;
57      /* Define a borda do input */
58      border: 1px solid #140042;
59      /* Arredonda os cantos do campo */
60      border-radius: 5px;
61  }
62
63  button {

```

```

C: > Users > Lucalhas > Downloads > # styles.css > ul
63  button {
64      /* Espaçamento interno do botão */
65      padding: 10px 20px;
66      /* Define a largura do botão */
67      width: 110px;
68      /* Define a altura do botão */
69      height: 40px;
70      /* Cor de fundo do botão */
71      background-color: #71a6df;
72      /* Cor do texto do botão */
73      color: white;
74      /* Remove a borda padrão */
75      border: none;
76      /* Arredonda os cantos */
77      border-radius: 5px;
78      /* Muda o cursor para formato de mão */
79      cursor: pointer;
80      /* Define a espessura da fonte */
81      font-weight: 400;
82  }
83
84  button:hover {
85      /* Altera a cor do botão quando o mouse passa sobre ele */
86      background-color: rgb(57, 142, 233);
87  }
88
89  p {
90      /* Define o tamanho da fonte */
91      font-size: 18px;
92      /* Deixa o texto em negrito */
93      font-weight: bold;
94      /* Define a cor do texto */

```

```

C: > Users > Lucalhas > Downloads > # styles.css > ul
89  p {
94      /* Define a cor do texto */
95      color: #a6342a;
96  }
97
98  ul {
99      /* Remove os marcadores da lista */
100     list-style: none;
101     /* Define a cor do texto da lista */
102     color: #aliceblue;
103     /* Centraliza o texto */
104     text-align: center;
105     /* Remove o espaçamento interno */
106     padding: 0;
107     /* Remove o espaçamento externo */
108     margin: 0;
109 }

```

Explicação Técnica

O código CSS é responsável pela estilização visual da aplicação, permitindo modificar cores, alinhamento, tamanhos, espaçamentos e efeitos gráficos dos elementos HTML. A folha de estilos trabalha separadamente da estrutura HTML, facilitando a organização e manutenção do projeto.

O seletor `body` define configurações gerais da página, como a fonte padrão e a cor de fundo. Além disso, utiliza o modelo Flexbox (`display: flex`) para centralizar todos os elementos horizontalmente e verticalmente na tela, enquanto `height: 100vh` faz o conteúdo ocupar toda a altura visível do navegador.

As classes `.container` e `.lista_container` estilizam as principais áreas da interface. Ambas utilizam `padding` para criar espaçamento interno, `border-radius` para arredondar os cantos e `box-shadow` para adicionar sombras, criando um efeito visual mais moderno e organizado.

Os seletores `input` e `button` personalizam os elementos interativos da aplicação. O campo de texto recebe bordas e espaçamento interno para melhorar a usabilidade, enquanto o botão possui tamanho fixo, cor personalizada e mudança de cursor para indicar interação. A pseudo-classe `button:hover` altera a cor do botão quando o mouse passa sobre ele, criando um efeito visual dinâmico.

Os elementos `p` e `ul` ajustam a exibição das mensagens e listas do sistema. O seletor `p` modifica tamanho, peso e cor da fonte, enquanto `ul` remove os marcadores padrões da lista e centraliza os itens exibidos na interface.

3 Código JavaScript

Raciocínio Passo a Passo:

1. A variável `terminado` é criada com valor `false`, indicando que o sistema ainda está funcionando.
2. A variável `lotacao` cria um vetor vazio para armazenar os nomes digitados.
3. A variável `mensagem` recebe o elemento HTML com id "mensagem" para permitir alterar mensagens na tela.
4. A variável `listaPessoas` recebe a lista HTML com id "lista" para adicionar os nomes validados.
5. A função `validarPessoas()` é executada quando o usuário clica no botão da página.
6. O código pega o valor digitado no campo de texto usando `document.getElementById("nome").value`.
7. O primeiro `if` verifica se o sistema já foi encerrado. Caso `terminado` seja verdadeiro, a função é interrompida com `return`.
8. O segundo `if` verifica se o campo está vazio. Se estiver, uma mensagem de erro é exibida em vermelho e a função é encerrada.
9. O terceiro `if` verifica se o usuário digitou "SAIR". Caso isso aconteça, o sistema informa que a lotação foi fechada e altera `terminado` para `true`.
10. Se nenhuma condição anterior acontecer, o nome é adicionado ao vetor `lotacao` usando `push(nome)`.
11. Depois disso, o sistema exibe uma mensagem de sucesso.
12. O código cria um novo elemento `<li>` usando `document.createElement("li")`.
13. O nome digitado é inserido dentro desse elemento usando `textContent`.
14. O item é adicionado à lista HTML através de `appendChild(linha)`.
15. O último `if` verifica se o vetor chegou a 7 nomes cadastrados. Caso isso aconteça, o sistema informa que a lotação está esgotada e encerra o programa alterando `terminado` para `true`.

Solução

```
C: > Users > Lucalhas > OneDrive > Área de Trabalho > IFSP > Técnico em Desenvolvimento de Sistemas > WEB > Bilheteria

1  var terminado = false;
2  /* Variável utilizada para controlar se o sistema foi encerrado */
3  var lotacao = [];
4  /* Vetor que armazenará os nomes cadastrados */
5  var mensagem = document.getElementById("mensagem");
6  /* Captura o elemento HTML responsável por exibir mensagens */
7  listaPessoas = document.getElementById("lista");
8  /* Captura o elemento HTML onde os nomes serão exibidos */
9
10 function validarPessoas() {
11     /* Função responsável por validar e cadastrar os nomes */
12     let nome = document.getElementById("nome").value;
13     /* Captura o valor digitado no campo de texto */
14     if(terminado){
15         /* Verifica se o sistema já foi encerrado */
16         return;
17         /* Encerra a função imediatamente */
18     }
19
20     if(nome === ""){
21         /* Verifica se nenhum nome foi digitado */
22         mensagem.textContent = "Insira um nome!";
23         /* Exibe mensagem de erro */
24         mensagem.style.color = "red";
25         /* Define a cor vermelha para a mensagem */
26         return;
27         /* Interrompe a execução da função */
28     }
29
30     if(nome === "SAIR"){
31         /* Verifica se o usuário digitou o padrão de saída */
32         mensagem.textContent = "Padrão de saída digitado! Lotação fechada!";
```

```

C: > Users > Lucalhas > OneDrive > Área de Trabalho > IFSP > Técnico em Desenvolvimento de Sistemas > WEB >
10  function validarPessoas(){
32      mensagem.textContent = "Padrão de saída digitado! Lotação fechada!";
33      /* Exibe mensagem informando encerramento */
34      mensagem.style.color = "black";
35      /* Define a cor da mensagem */
36      terminado = true;
37      /* Marca o sistema como encerrado */
38      return;
39      /* Finaliza a função */
40  }
41
42  lotacao.push(nome);
43  /* Adiciona o nome digitado ao vetor */
44  mensagem.textContent = "Validação de nome feita com sucesso!";
45  /* Exibe mensagem de sucesso */
46  mensagem.style.color = "black";
47  /* Define a cor da mensagem */
48
49  let linha = document.createElement("li");
50  /* Cria um elemento de lista */
51  linha.textContent = nome;
52  /* Define o nome digitado como conteúdo do item */
53  listaPessoas.appendChild(linha);
54  /* Adiciona o item criado à lista exibida na página */
55
56  if(lotacao.length === 7){
57      /* Verifica se a quantidade de inscritos chegou a 7 */
58      mensagem.textContent = "Lotação esgotada! Temos 7 inscritos.";
59      /* Exibe mensagem informando lotação máxima */
60      mensagem.style.color = "black";
61      /* Define a cor da mensagem */
62      terminado = true;

```

```

C: > Users > Lucalhas > OneDrive > Área de Trabalho > IFSP > Técnico em Desen
10  function validarPessoas(){
61      /* Define a cor da mensagem */
62      terminado = true;
63      /* Encerra o sistema de cadastro */
64      return;
65      /* Finaliza a execução da função */
66  }
67  }

```

Explicação Técnica

O código em JavaScript implementa um sistema simples de controle de lotação, permitindo validar nomes digitados pelo usuário e armazená-los em uma lista dinâmica exibida na página.

A variável `terminado` funciona como um controle lógico do sistema. Quando seu valor é `true`, a aplicação deixa de aceitar novos nomes. Já o vetor `lotacao` é utilizado para armazenar todos os nomes cadastrados durante a execução do programa.

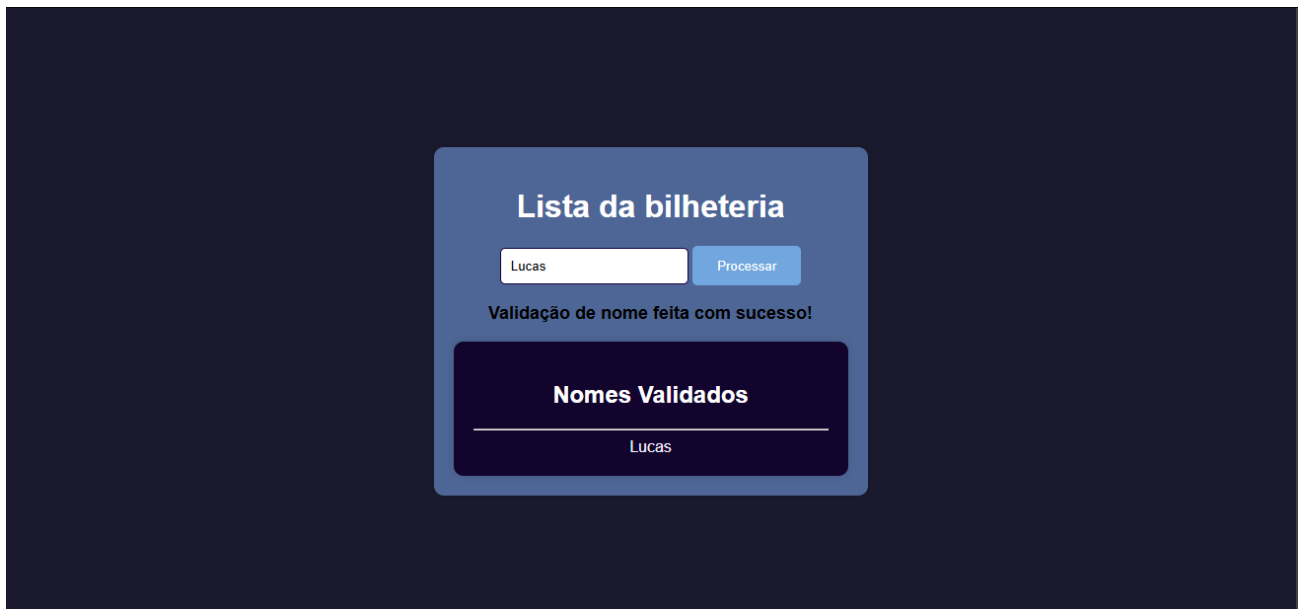
Os comandos `document.getElementById()` são usados para acessar elementos HTML através de seus identificadores. Isso permite que o JavaScript altere mensagens na tela e adicione itens dinamicamente na lista exibida ao usuário.

A função `validarPessoas()` concentra toda a lógica do sistema. Ela captura o valor digitado no campo de texto e utiliza estruturas condicionais `if` para validar diferentes situações, como campo vazio, encerramento manual do sistema e limite máximo de participantes.

O método `push()` adiciona novos nomes ao vetor `lotacao`, enquanto `document.createElement("li")` cria elementos HTML dinamicamente. O método `appendChild()` insere esses elementos na lista da página, permitindo atualização em tempo real da interface.

O uso da propriedade `textContent` altera textos de forma segura dentro dos elementos HTML, enquanto `style.color` modifica dinamicamente a cor das mensagens exibidas ao usuário. Dessa forma, o sistema combina manipulação do DOM, estruturas condicionais e vetores para criar uma interface interativa e funcional.

Print Screen



Conclusão

A atividade permitiu compreender, na prática, a importância da auditoria e da integração de sistemas durante o desenvolvimento de aplicações. Inicialmente, foi necessário identificar e corrigir erros lógicos e estruturais presentes no código da bilheteria, garantindo o funcionamento correto do laço de repetição e das estruturas condicionais.

Após a correção da lógica, o sistema foi integrado a uma interface gráfica utilizando HTML, CSS e JavaScript. O HTML foi responsável pela estrutura da página, o CSS pela estilização visual e o JavaScript pela implementação da lógica e da interação com o usuário.

Além disso, o projeto demonstrou conceitos importantes da programação, como manipulação de vetores, estruturas condicionais, laços de repetição, funções, eventos e manipulação do DOM. A criação dinâmica da lista de convidados mostrou como o JavaScript pode atualizar elementos da página em tempo real, tornando a aplicação mais interativa e funcional.

Dessa forma, a atividade contribuiu para o desenvolvimento de habilidades relacionadas à correção de erros, organização de código, integração entre tecnologias web e construção de interfaces funcionais, aspectos fundamentais no desenvolvimento de sistemas modernos.

Referências Bibliográficas

GOOGLE. Gemini: modelo de inteligência artificial generativa para suporte pedagógico e revisão de código. Mountain View, 2026. Disponível em: <https://gemini.google.com>. Acesso em: 09 mai. 2026.

OPENAI. ChatGPT: modelo de inteligência artificial generativa para suporte pedagógico e revisão de código. San Francisco, 2026. Disponível em: <https://chat.openai.com>. Acesso em: 09 mai. 2026.